Contexto del proyecto: ablación láser con círculos y anillos

Documento de traspaso para quien continúe el trabajo. Proyecto: Laboratorio Avanzado (MATLAB + motores OWIS PS35 + láser).

1. Resumen en pocas líneas

Existe un sistema en MATLAB (PanelOwis) que recibe una imagen en blanco y negro, detecta las esquinas de las figuras y mueve una mesa con motores OWIS para que el láser ablacione esas figuras. Ese sistema solo sabe trabajar con **rectángulos** (figuras de 4 vértices) y rellenarlos con líneas rectas. El objetivo del proyecto es que funcione con **círculos y anillos**: detectar centros y radios en la imagen y trazarlos con interpolación circular del controlador PS35 (círculos aproximados con secantes).

2. Cómo funcionaba el sistema original

Hardware y librerías

- Controlador OWIS PS35 con dos ejes (X = eje 1, Y = eje 2), manejado desde MATLAB con `loadlibrary('ps35')` y `calllib` (cabecera `ps35.h`).
- El láser se enciende y apaga con una línea digital de una tarjeta NI mediante `DAQstate.m` (Dev2, Port1/Line3). Ojo: la lógica está invertida (`DAQstate(true)` escribe 0 en la línea y `DAQstate(false)` escribe 1).
- Add-ons de MATLAB requeridos (según el Reedme): soporte MinGW-w64, Arduino, NI DAQ Toolbox, Data Acquisition Toolbox.

Archivos principales (todos deben estar en la misma carpeta)

- `PanelOwis.m / .fig`: interfaz gráfica (GUIDE). Botones relevantes: conectar (`pushbutton5`), cargar imagen, *Vertex Detection* (`pushbutton9`, solo visualiza las esquinas) y *Go Ablation* (`pushbutton7`), además de mover, home, pausa, stop, disconnect y repetición de pulsos.
- `VertexDetection.m`: lee el PNG, lo convierte a binario y, por cada región blanca, busca 4 esquinas con `pgonCorners.m` (redondeadas a múltiplos de 5 px). Solo sirve para polígonos convexos de 4 vértices.
- `generalMov.m`: por cada figura toma el rectángulo que la contiene, mueve la mesa con el láser apagado al inicio y llama a `ps35New` para rellenarla con líneas largas en X (n=1) o en Y (n=2) separadas por el paso (*step-size*).

- `ps35New.m`, `ps35tool.m`, `ps35toolX/Y(.m)`, `ps35toolX2/Y2`: funciones de movimiento recto sobre uno o dos ejes.
- `repRate.m`, `twotr.m`, `twotrun.m`, `unipolar*.m`: modo de repetición de pulsos (puntos) con salida analógica NI.
- `Reedme.txt / .docx`: instrucciones de instalación y operación del panel.

Convención de la imagen (manual de Inkscape incluido): dibujo en blanco sobre fondo negro, unidades en px, escala 1.0, exportado como PNG sin características especiales. En `generalMov` 1 píxel equivale a 1 µm (se divide entre 1000 para pasar a mm). Eje X del motor = columna de la imagen, eje Y = fila.

3. Qué se quiere lograr

1. Reemplazar `VertexDetection` por una función que detecte **centros y radios** de todos los círculos de la imagen.
2. Poder trazar a veces **círculos sencillos** y a veces **anillos** (círculos concéntricos rellenos).
3. Unir esa detección con el trazado de círculos con secantes (`PS35_CircleToVectorTab`), que ya se había trabajado en los `LabAdv_Motors_*_TableScript.m`.
4. Que todo funcione desde el PanelOwis.

Condición del profesor: no crear archivos nuevos con los mismos nombres que los originales, para no confundirlos. Las versiones modificadas (PanelOwis, RingDetection, etc.) llevan otro nombre y la versión de círculos se guarda en una **carpeta aparte** de la original.

4. Trabajo previo con secantes (antecedente)

Los scripts `LabAdv_Motors_25_10_29` hasta `25_11_27_TableScript.m` son la línea de trabajo anterior con la tabla de vectores del PS35: construyen cadenas de renglones con `PS35_CircleToVectorTab` (formato de 10 parámetros: ejes, tiempo por secante, código de función, número de secantes, radio circunscrito en incrementos, ángulo inicial, ángulo del arco y factores de escala). Evolucionaron desde un circuito sencillo hasta ramas superior/inferior con radios distintos (R1/R2), colas rectas izquierda/derecha y secciones tipo embudo. Son scripts sueltos que dependen de variables definidas fuera (`axis_X`, `motorSpeed`, `mm2incrementsFactor`, `plate_Length`, etc.). `ps35tool_Circ_Antonio.m` es una prueba pequeña de la misma idea. Límites importantes del PS35: máximo de ~1.638 s por secante y tabla de vectores limitada (aquí se usa un tope de 2000 renglones).

5. Pasos realizados

1. Se definió el plan: primero encontrar una función de detección de centros y radio que funcione dentro de PanelOwis.
2. Se decidió que debía detectar **todos** los círculos de la imagen y permitir trazar círculos sencillos y anillos.
3. Se logró una nueva función de detección que también detecta **anillos (círculos concéntricos)**, separada de la `CircleDetection` original. Es `RingDetection`: entrega `res` (un `[cy cx r]` por cada borde detectado) y `grupos` (qué bordes son concéntricos).
4. Se decidió conectarla **primero con `circulo_while`** (la función que ya mueve los motores para hacer un círculo), antes de trabajar con las secantes por separado.
5. Se convirtió `circulo_while` (versión del 2026-05-28) en una función llamada desde el panel: **`anilloMov.m`**.
6. Se escribió **`circlesMov.m`**, el equivalente de `generalMov` para círculos y anillos.

6. Estado actual del código nuevo (carpeta `claude/`)

`anilloMov(R_int, R_ext, p)`

- Traza un círculo (si R_int ≈ R_ext) o un anillo relleno con círculos concéntricos separados por el paso del láser.
- Usa interpolación circular del PS35 (`PS35_CircleToVectorTab`, `PS35_GoVectorTab`) en **modo relativo**: el controlador no conoce el centro. El círculo arranca donde esté la mesa, que debe estar en el ángulo 90° del primer círculo, es decir en `(cx, cy + R_int)`, y termina en `(cx, cy + R_final)`.
- Diferencias con el live script original: no hace `clear/close all/clc`, no carga ni descarga la librería ni se conecta/desconecta (usa la conexión ya abierta), parte en lotes si se pasa de 2000 renglones, y avisa si alguna llamada al PS35 regresa error.
- Parámetros en la estructura `p` (con valores por defecto): `axis_X=1`, `axis_Y=2`, `motorSpeed=0.2e-3` mm/ms, `laserStepWidth=0.7e-3` mm, `secantNum=100`, `angleStart=90`, `t_max=1600` ms, `mm2incrementsFactor=10000`, `laserOn=true`, `modoPaso='tabla'` (todo seguido) o `'separado'` (pieza por pieza), `tPasoMin=20` ms, `verbose`.
- `laserOn=false` permite **trazar sin prender el láser**, útil para pruebas.

`circlesMov(res, grupos, BW, p)`

- Para cada grupo de círculos: si es uno solo, lo traza como círculo sencillo. Si son varios concéntricos, los recorre de menor a mayor radio y revisa en la imagen `BW` si entre dos radios consecutivos hay blanco (→ anillo relleno) o negro (→ dos líneas separadas, cada una un círculo sencillo).
- Por cada trazo: mueve la mesa en línea recta con láser apagado al punto de arranque, llama a `anilloMov` y al final regresa al punto de inicio.
- Si `p.COMp` viene definido, se conecta al PS35 dentro de la propia función (así no depende de haber presionado el botón de conectar).
- Imprime el plan de trazos antes de mover.
- **Cambio importante de convención:** el origen ahora es el **centro de la imagen**, no la esquina como en `generalMov`. La posición de la mesa al presionar *Run* debe corresponder al centro de la imagen. Escala por defecto `p.pix2mm = 1/1000` (1 px = 1 µm).

7. Lo que falta / pendientes

(Esta sección son sugerencias de continuación, no decisiones tomadas.)

- **Integrar en el panel:** `pushbutton7` (Go Ablation) todavía llama a `VertexDetection` y `generalMov`. Hay que hacer una copia del panel con otro nombre (por la regla del profesor) que llame a `RingDetection` + `circlesMov`, armando `p` con COM, velocidades, ejes, escala y paso. Lo mismo con el botón de visualización (`pushbutton9`), que hoy dibuja esquinas.
- **Pruebas en el equipo:** no está documentado en estos archivos que `anilloMov` / `circlesMov` ya se hayan corrido en la mesa real. Se recomienda probar primero con `laserOn=false` y con un solo círculo pequeño, luego un anillo, luego varios.
- **Verificar unidades:** que `motorSpeed`, `laserStepWidth` y el factor `mm2incrementsFactor` correspondan a la calibración real de la mesa, y que 1 px = 1 µm siga siendo válido para las imágenes usadas.
- **Coordinar el origen:** decidir y comunicar a quien opere que el punto de partida es el centro de la imagen.
- **Validar el criterio anillo vs. líneas separadas** (muestreo de 72 puntos entre radios) con imágenes reales.

8. Cosas a tener en cuenta del código heredado

- El botón de pausa del panel solo cambia el LED y el texto; no detiene realmente el movimiento.
- La lógica del láser en `DAQstate` está invertida (ver sección 2) y las tarjetas NI cambian entre archivos (`Dev2` en `DAQstate` , `Dev3` en `repRate` y las pruebas de pulsos).
- La velocidad de comunicación serial aparece con valores distintos (9600, 115200, 230400) según el archivo; el panel y los archivos nuevos usan 230400.
- Varias funciones antiguas (`ps35New` , `repRate`) cargan y descargan la librería `ps35` en cada llamada; si se mezclan con `anilloMov` / `circlesMov` (que asumen la librería ya cargada y conectada) puede haber conflictos.
- En el Reedme hay dos puntos numerados como 11.
- Los archivos `RingDetection.m` y `circulo_while` (live script original) no estaban entre los archivos que se revisaron para este documento; conviene adjuntarlos al traspaso.

9. Archivos a entregar a la persona que continúe

- Carpeta original (sistema de esquinas) **sin modificar**.
- Carpeta aparte con la versión de círculos: `RingDetection.m` , `anilloMov.m` , `circlesMov.m` , copia renombrada de PanelOwis, `DAQstate.m` , `ps35.h` / `ps35.dll` .
- `circulo_while` (original, versión 2026-05-28), el Reedme y el manual de Inkscape.
- Los `LabAdv_Motors_*_TableScript.m` como referencia del trabajo con secantes.